4주차 4일차: 작은 WPF API 테스트 프로그램 완성 — 120분

1. 학습 목표 — 5분

- 입력 URL → Command → 비동기 GET → 데이터 변환 → 표 표시를 연결합니다.
- 응답 항목의 번호와 제목을 최소한으로 확인합니다.
- 새 기능을 늘리지 않고 최종 프로그램을 완성합니다.

시작 상태는 day15입니다. 최종 화면은 입력창 하나, 실행 버튼 하나, 결과 표, 결과 건수와 상태 문구입니다.

2. 핵심 개념 설명 — 15분

지난 시간의 코드는 JSON 배열이면 Post로 읽어 옵니다. 그런데 전혀 다른 데이터의 배열인 [{}]도 기본값이 들어 있는 Post가 될 수 있습니다. 형식을 객체 목록으로 읽었다는 사실만으로 학습용 게시글 데이터라고 보장되지는 않습니다.

최종 코드에는 각 항목의 Id가 양수이고 Title이 비어 있지 않은지 확인하는 작은 조건문을 추가합니다. 하나라도 맞지 않으면 표에 넣기 전에 JsonException으로 처리합니다. 이는 이 교재의 게시글 데이터 계약에 맞춘 최소 확인이며 모든 API의 공통 검증 규칙은 아닙니다. UserId와 Body는 JSON 매핑을 이해하기 위해 보관하지만 화면에는 표시하지 않습니다.

정상 응답이 빈 배열이면 반복문을 건너뛰어 결과 0건을 표시합니다. null은 배열이 아니므로 형식 오류로 처리합니다. 항목 검사를 먼저 마친 뒤 행을 추가하므로 검사 실패 시 일부 행만 남지 않습니다.

```
TextBox → Input
Button → RunCommand → RunAsync
URL 확인 → HttpClient GET → List<Post>
항목 확인 → ObservableCollection<Post> → DataGrid
오류 발생 → catch → Status → 화면 안내
```

입력창과 결과를 잇는 Binding, DataContext에 연결한 ViewModel, 속성과 목록의 변경 알림, 비동기 요청이 처음부터 끝까지 이어졌습니다. 1주차와 비교해도 화면 구성은 간단하게 유지합니다.

3. 짧고 이해하기 쉬운 코드 예제 — 15분

```
        // 항목에 번호와 제목이 있는지 최소한으로 확인합니다.
        foreach (Post post in posts)
        {
            if (post is null || post.Id <= 0 || string.IsNullOrWhiteSpace(post.Title))
            {
                throw new JsonException("번호와 제목이 있는 게시글 배열이 필요합니다.");
            }
        }
```

이 검사 다음에 목록을 채우고 건수를 표시합니다.

```
        foreach (Post post in posts)
        {
            Posts.Add(post);
        }

        Result = $"결과 {Posts.Count}건";
        Status = "GET 요청을 완료했습니다.";
```

전체 파일은 MainViewModel.cs 링크에 있습니다. 오늘 추가하는 기능은 이 검사뿐이며 새 화면이나 패키지는 없습니다.

오늘 바뀌는 파일입니다. 본문은 핵심 부분만 보여 주므로 작성·교체에는 링크의 **전체 코드**를 사용합니다.

파일	전체 코드
MainViewModel.cs	day16의 MainViewModel.cs

4. Visual Studio에서 따라 하는 실습 — 75분

최종 코드 적용 — 20분

1. 창을 닫고 MainViewModel.cs를 오늘의 전체 코드로 교체합니다. 다른 파일은 day15와 같습니다.
2. 요청 → null 확인 → 항목 확인 → 행 추가 → 성공 안내의 순서를 찾습니다.
3. 솔루션을 빌드하고 F5로 실행합니다. 기본 URL로 결과가 표시되는지 확인합니다.

정상 결과와 빈 결과 — 20분

1. https://jsonplaceholder.typicode.com/posts?userId=1 로 조회합니다.
2. userId=2로 바꾸어 다른 결과가 표시되는지 확인합니다.
3. userId=999999로 바꾸어 조회합니다. 현재 학습용 API에서는 []를 반환하므로 "결과 0건"과 완료 상태가 표시됩니다.
4. 기본 URL로 되돌립니다. 실제 서버 데이터나 제공 정책이 바뀌면 공식 API 안내를 기준으로 확인합니다.

오류 후 다시 실행 — 15분

1. "잘못된주소"를 실행하여 URL 안내를 확인합니다.
2. https://jsonplaceholder.typicode.com/posts/1 로 실행하여 단일 객체 형식 오류를 확인합니다.
3. 기본 URL로 다시 조회해 정상적으로 사용할 수 있는지 확인합니다.
4. API 응답이 빠르면 비활성화가 잠깐만 보일 수 있습니다. 비동기 대기는 day13의 2초 예제에서 이미 확인했습니다. 별도 인위적 지연은 최종 코드에 넣지 않습니다.

전체 흐름 확인과 저장 — 20분

1. XAML의 Input, RunCommand, Posts, Result, Status를 찾습니다.

2. MainWindow.xaml.cs의 DataContext=new MainViewModel()을 찾습니다.
3. MainViewModel에서 입력 URL을 사용하는 줄, await 줄, 결과 목록을 채우는 줄을 차례로 봅니다.
4. 중단점이 남았다면 제거하고 저장합니다.
5. 창을 닫았다 다시 실행해 기본 URL로 한 번 조회합니다. 결과를 파일이나 데이터베이스에 저장하지 않으므로 이전 조회 결과는 남지 않습니다.

별도의 과제나 배포 절차는 없습니다. 이 프로그램을 작은 테스트 창을 만들어 본 학습 경험으로 남기면 됩니다.

진도가 다르면 저장소 루트에서 powershell -ExecutionPolicy Bypass -File .₩sample₩Use-Stage.ps1 -Stage 15로 시작 상태를 적용합니다. 오늘의 완료본이 필요하면 Stage를 16로 지정합니다. 실행 중인 앱을 먼저 닫고 변경 파일을 다시 불러옵니다. 단계 적용·백업 안내

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 10분

- XAML은 화면을, ViewModel은 화면 상태와 동작을, Post는 응답 데이터 모양을 맡습니다.
- Binding과 알림으로 C# 데이터 변경이 화면에 반영됩니다.
- Command로 실행을 연결하고 await로 요청 대기 중 화면 반응을 유지합니다.
- 결과 표는 예상한 게시글 배열을 위한 것이며 다른 API에는 모델과 매핑 수정이 필요합니다.
- 실무·면접에서는 직접 만든 이 흐름을 설명할 수 있으면 됩니다. 고급 설계나 WPF 전체 기능을 암기할 필요는 없습니다.

완료본: day16. 전체 학습 안내로 돌아가기.